MASTER PROMPT — HIGH-LEVEL SYSTEM DESIGN / SYSTEM DESIGN INTERVIEW ROADMAP

My Background

I am a Software Engineer / Tech Lead with:

- **9 years of professional experience**
- Strong backend development experience
- PHP / Laravel
- Node.js
- JavaScript
- MySQL
- PostgreSQL
- MongoDB
- Redis
- Kafka
- Elasticsearch
- GraphQL
- Docker
- AWS
- Kubernetes / EKS / ECS
- Terraform
- Prometheus
- Grafana
- OpenTelemetry
- Microservices
- SaaS architecture

I have practical software engineering experience, but I am **relatively new to High-Level System Design (HLD) / System Design interviews**.

My goal is to become strong enough in System Design that I can confidently crack **Tier-2 to Tier-1 product-based companies in India and Dubai/UAE**, including companies with interview standards similar to:

- Amazon
- Google
- Microsoft
- Meta
- Uber
- Careem
- Noon
- Talabat
- Deliveroo

- Stripe
- Airbnb
- Flipkart
- Walmart
- Atlassian
- Razorpay
- PhonePe
- CRED
- Meesho
- Swiggy
- Zomato
- Zepto
- and other strong product companies.

Do NOT assume that I already understand System Design concepts just because I have 9 years of development experience.

I want to learn from **fundamentals → intermediate → advanced → interview-ready**.

PRIMARY OBJECTIVE

Create a **complete High-Level System Design learning and interview-preparation program** that I can follow almost blindly.

The roadmap must teach me:

1. What System Design actually is
2. Why each concept exists
3. When to use it
4. When NOT to use it
5. How different components communicate
6. How to make architecture decisions
7. How to calculate capacity
8. How to identify bottlenecks
9. How to scale systems
10. How to design reliable distributed systems
11. How to handle failures
12. How to discuss trade-offs
13. How to communicate architecture during interviews
14. How to approach unknown system-design questions
15. How to design real-world products from scratch
16. How to defend my architecture when the interviewer challenges it.

The final objective is:

Give me the ability to enter a System Design interview with an unknown product/ problem and systematically design a production-grade scalable system.

IMPORTANT: RESEARCH REQUIREMENT

Before creating the roadmap, perform thorough research using current internet resources.

Prioritize:

Official / High-quality resources

- AWS Architecture Center
- AWS documentation
- Google Cloud Architecture documentation
- Microsoft Azure Architecture Center
- Martin Kleppmann resources
- Designing Data-Intensive Applications
- System Design Primer
- High Scalability
- ByteByteGo
- Educative
- AlgoMaster
- GeeksforGeeks
- InterviewBit
- Grokking System Design
- Engineering blogs from Amazon
- Engineering blogs from Uber
- Engineering blogs from Netflix
- Engineering blogs from Meta
- Engineering blogs from Google
- Engineering blogs from Microsoft
- Engineering blogs from Airbnb
- Engineering blogs from Stripe
- Engineering blogs from Cloudflare
- Engineering blogs from LinkedIn
- Engineering blogs from Discord
- Engineering blogs from DoorDash
- Engineering blogs from Careem where available.

Also research **recent interview experiences and system-design questions** for:

- Amazon India
- Amazon UAE/Dubai
- Careem
- Noon

- Talabat
- Uber
- Google
- Microsoft
- Flipkart
- Walmart
- Atlassian
- Razorpay
- PhonePe
- CRED
- Swiggy
- Zomato
- Meesho.

Use recent information wherever possible.

Clearly separate:

- Evergreen concepts
- Current interview trends
- Company-specific interview expectations.

PART 1 — CREATE THE COMPLETE LEARNING ROADMAP

Create a structured roadmap from:

LEVEL 0 — Prerequisites

Teach only the prerequisites actually required for HLD.

Include:

- HTTP
- HTTPS
- TCP
- UDP
- DNS
- TLS
- IP
- Ports
- REST
- gRPC
- WebSockets

- API Gateway
- Reverse Proxy
- Load Balancer
- CDN
- Cookies
- Sessions
- JWT
- OAuth
- Authentication
- Authorization
- Basic networking concepts.

Explain each in simple language.

LEVEL 1 — SYSTEM DESIGN FUNDAMENTALS

Teach:

- What is System Design?
- HLD vs LLD
- Functional requirements
- Non-functional requirements
- Scalability
- Availability
- Reliability
- Durability
- Performance
- Latency
- Throughput
- Consistency
- Fault tolerance
- Maintainability
- Observability
- Security
- Cost
- CAP theorem
- PACELC
- Horizontal scaling
- Vertical scaling
- Stateless vs stateful services
- Monolith
- Modular monolith
- Microservices
- Service-oriented architecture.

For every topic provide:

- Simple explanation
- Real-world analogy
- Technical explanation
- When to use
- When not to use
- Advantages
- Disadvantages
- Interview questions
- Real-world examples.

LEVEL 2 — CAPACITY ESTIMATION

Teach capacity estimation from absolute basics.

Cover:

- Users
- DAU
- MAU
- Concurrent users
- Requests per second
- Peak RPS
- Average RPS
- Read/write ratio
- Storage estimation
- Bandwidth estimation
- Memory estimation
- Cache size
- Database size
- Growth estimation
- Replication factor
- Peak traffic
- Traffic multiplier
- QPS calculations.

Teach a reusable interview framework.

Provide at least 10 worked examples.

Examples:

- URL Shortener
- WhatsApp

- Instagram
- Food Delivery
- E-commerce
- Ride Sharing
- Payment System
- Video Streaming
- Notification System
- File Storage.

LEVEL 3 — CORE BUILDING BLOCKS

Create a complete chapter for every major building block.

Load Balancer

Cover:

- L4 vs L7
- Algorithms
- Round Robin
- Least Connections
- IP Hash
- Consistent Hashing
- Health Checks
- Sticky Sessions
- Failover
- Scaling.

CDN

Cover:

- Edge locations
- Origin
- Cache hit/miss
- TTL
- Cache invalidation
- Static content
- Dynamic content
- CDN architecture.

Reverse Proxy

Explain:

- Nginx
- HAProxy
- Envoy
- Reverse proxy vs load balancer.

API Gateway

Cover:

- Routing
- Authentication
- Rate limiting
- Throttling
- Request transformation
- Logging
- Versioning.

Cache

Deeply cover:

- Redis
- Memcached
- Cache-aside
- Read-through
- Write-through
- Write-back
- Write-around
- TTL
- Eviction
- LRU
- LFU
- Cache stampede
- Cache penetration
- Cache avalanche
- Hot keys
- Distributed cache
- Cache invalidation.

Databases

Teach:

SQL

- MySQL
- PostgreSQL
- ACID
- Transactions
- Indexes
- Composite indexes
- Query optimization
- Joins
- Replication
- Read replicas
- Partitioning
- Sharding.

NoSQL

- MongoDB
- DynamoDB
- Cassandra
- Redis.

Explain:

- When SQL?
- When NoSQL?
- When MongoDB?
- When DynamoDB?
- When Cassandra?
- How to choose a database in an interview?

LEVEL 4 — DATABASE DEEP DIVE

Cover:

- Primary key
- Secondary index
- B-Tree
- Hash index
- Clustered index
- Non-clustered index
- Transactions
- Isolation levels
- Locks
- Deadlocks
- MVCC

- Replication
- Leader/follower
- Multi-leader
- Leaderless
- Read-after-write consistency
- Eventual consistency
- Strong consistency
- Sharding
- Horizontal partitioning
- Vertical partitioning
- Consistent hashing
- Hot partitions
- Rebalancing.

Explain database scaling step-by-step:

1 database → read replica → cache → partitioning → sharding → distributed database.

LEVEL 5 — DISTRIBUTED SYSTEMS

This must be one of the deepest sections.

Cover:

- Distributed systems fundamentals
- Network failures
- Partial failures
- Timeouts
- Retries
- Exponential backoff
- Jitter
- Circuit breaker
- Bulkhead
- Rate limiting
- Backpressure
- Idempotency
- Distributed locking
- Leader election
- Consensus
- Quorum
- Replication
- Eventual consistency
- Strong consistency
- Ordering
- Duplicate messages

- Exactly-once vs at-least-once vs at-most-once
- Clock problems
- Distributed transactions
- Saga
- 2PC
- Outbox pattern
- CDC
- CQRS
- Event sourcing.

For every concept provide a practical example.

LEVEL 6 — MESSAGE QUEUES / EVENT-DRIVEN ARCHITECTURE

Deeply explain:

- Message Queue
- Pub/Sub
- Kafka
- RabbitMQ
- SQS
- SNS
- Kinesis.

Kafka must be covered deeply:

- Producer
- Consumer
- Topic
- Partition
- Offset
- Consumer group
- Replication
- Leader
- Follower
- Ordering
- Retention
- Rebalancing
- Delivery semantics
- Dead Letter Queue
- Retry topics
- Backpressure.

Explain when to use:

- Kafka
- RabbitMQ
- SQS
- SNS
- Redis Streams.

LEVEL 7 — MICROSERVICES

Cover:

- Monolith
- Modular monolith
- Microservices
- Service boundaries
- Database-per-service
- API communication
- Sync communication
- Async communication
- Service discovery
- API Gateway
- Configuration
- Secrets
- Distributed tracing
- Health checks
- Deployment
- Versioning
- Backward compatibility.

Also cover:

- Microservices failure modes
- Distributed transactions
- Service dependency problems
- Cascading failures.

LEVEL 8 — API DESIGN

Teach:

- REST
- GraphQL

- gRPC
- WebSockets
- API versioning
- Pagination
- Cursor pagination
- Offset pagination
- Filtering
- Sorting
- Idempotency keys
- Rate limiting
- API authentication
- Authorization
- Error handling
- Retry-safe APIs.

Show good and bad API designs.

LEVEL 9 — SECURITY

Cover System Design security:

- Authentication
- Authorization
- OAuth2
- JWT
- RBAC
- ABAC
- API keys
- TLS
- Encryption at rest
- Encryption in transit
- Secrets management
- Password hashing
- SQL injection
- XSS
- CSRF
- SSRF
- DDoS
- WAF
- Rate limiting
- Zero Trust
- Audit logging
- PII protection.

Explain how security should appear in architecture diagrams.

LEVEL 10 — OBSERVABILITY & OPERATIONS

Cover:

- Logging
- Metrics
- Tracing
- OpenTelemetry
- Prometheus
- Grafana
- ELK
- Distributed tracing
- Correlation ID
- SLI
- SLO
- SLA
- Alerting
- Health checks
- Monitoring
- Incident response
- Disaster recovery
- Backup
- Restore
- RPO
- RTO
- Multi-AZ
- Multi-region.

LEVEL 11 — CLOUD SYSTEM DESIGN

Since I have AWS experience, use AWS as the primary cloud.

Cover:

- EC2
- ECS
- EKS
- Lambda
- API Gateway
- ALB
- NLB
- CloudFront
- S3
- RDS

- Aurora
- DynamoDB
- ElastiCache
- MSK
- SQS
- SNS
- Route 53
- WAF
- IAM
- KMS
- CloudWatch.

For every AWS service explain:

- What problem it solves
- When to use it
- When NOT to use it
- Alternatives
- Interview usage
- Cost/scaling considerations.

LEVEL 12 — ARCHITECTURE PATTERNS

Deeply explain:

- Load balancing
- Cache-aside
- Read-through cache
- Write-through
- Write-back
- CQRS
- Event sourcing
- Saga
- Outbox
- CDC
- Pub/Sub
- Event-driven architecture
- Fan-out
- Bulkhead
- Circuit breaker
- Retry
- Rate limiting
- Token bucket
- Leaky bucket
- Consistent hashing

- Sharding
- Leader election
- Strangler pattern
- Sidecar
- Ambassador
- Service mesh.

For each:

1. Problem
2. Solution
3. Architecture
4. Example
5. Advantages
6. Disadvantages
7. When to use
8. When not to use
9. Interview questions.

LEVEL 13 — DESIGN INTERVIEW FRAMEWORK

Create a **universal framework** that I can memorize and use for ANY System Design question.

For example:

Step 1 — Clarify Requirements

Functional requirements:

- What does the system do?

Non-functional requirements:

- Scale
- Availability
- Latency
- Consistency
- Security
- Durability.

Step 2 — Estimate Scale

Calculate:

- DAU

- RPS
- Peak RPS
- Storage
- Bandwidth.

Step 3 — Define APIs

Step 4 — Define Data Model

Step 5 — Create High-Level Architecture

Step 6 — Deep Dive

Discuss:

- Database
- Cache
- Queue
- Scaling
- Consistency
- Failure handling
- Security.

Step 7 — Bottlenecks

Identify:

- Database bottleneck
- Cache bottleneck
- Network bottleneck
- CPU bottleneck
- Hot partition
- Queue backlog.

Step 8 — Reliability

Discuss:

- Failover
- Retry
- Timeout
- Circuit breaker
- Replication
- Disaster recovery.

Step 9 — Trade-offs

Step 10 — Future Scaling

Teach me exactly **what to say verbally in each step during an interview.**

LEVEL 14 — HOW TO DRAW SYSTEM DESIGN DIAGRAMS

Teach me how to draw clean diagrams.

Show standard notation for:

- Client
- CDN
- Load balancer
- API Gateway
- Services
- Cache
- Database
- Queue
- Workers
- Object storage
- External services.

Teach:

- Request flow
- Data flow
- Async flow
- Failure flow.

Provide examples of good interview diagrams.

LEVEL 15 — COMPLETE SYSTEM DESIGN PROBLEMS

Create progressively difficult problems.

Beginner

1. URL Shortener

2. Pastebin
3. Rate Limiter
4. File Upload System
5. Notification Service
6. Logging System

Intermediate

1. Twitter/X
2. Instagram
3. WhatsApp
4. YouTube
5. Dropbox
6. Google Drive
7. E-commerce System
8. Food Delivery System
9. Hotel Booking
10. Ticket Booking

Advanced

1. Uber
2. Careem
3. Amazon
4. Netflix
5. Payment System
6. Wallet System
7. Ride Matching
8. Food Delivery at Scale
9. Search System
10. Recommendation System
11. Ad Serving System
12. Distributed Notification System
13. Real-time Chat
14. Video Streaming.

For EVERY problem provide:

1. Requirements

2. Capacity estimation

3. APIs

4. Data model

5. Architecture

6. Request flow

7. Database choice

8. Cache strategy

9. Queue strategy

10. Scaling strategy

11. Failure handling

12. Consistency

13. Security

14. Observability

15. Bottlenecks

16. Trade-offs

17. Follow-up questions

18. Interviewer challenges

19. Ideal candidate answers

20. 45-minute interview version.

LEVEL 16 — COMPANY-SPECIFIC PREPARATION

Create separate System Design expectations for:

Tier 1

- Amazon
- Google
- Microsoft
- Meta
- Uber
- Atlassian
- Airbnb.

India Product Companies

- Flipkart
- Walmart
- Razorpay
- PhonePe
- CRED
- Meesho
- Swiggy
- Zomato
- Zepto.

Dubai/UAE

- Careem
- Noon
- Talabat
- Deliveroo
- Tabby
- Tamara
- PayBy
- Ziina
- Magnati
- Emirates-related technology/product organizations.

For each company explain:

- Expected level
- Typical System Design round
- HLD vs LLD expectation
- Common questions
- Important domains
- Difficulty
- What interviewer evaluates
- Typical follow-up questions
- What makes an answer strong.

Clearly mention when information is based on publicly reported interview experiences versus official company information.

LEVEL 17 — FINTECH SYSTEM DESIGN

Because I am interested in FinTech/product companies, create a dedicated module covering:

- Payment gateway
- Payment processing
- Wallet
- Ledger
- Double-entry accounting
- Transaction processing
- Idempotency
- Payment retries
- Refund
- Chargeback
- Fraud detection
- Payment reconciliation
- Webhooks
- Payment state machine
- Distributed transaction
- Saga
- Event-driven payment architecture.

Explain these at interview depth.

LEVEL 18 — REAL-TIME SYSTEM DESIGN

Cover:

- WebSockets
- SSE
- Long polling
- Pub/Sub
- Presence
- Real-time location
- Real-time tracking
- Chat
- Notifications
- Live dashboards
- Ride tracking.

Include scaling strategies.

LEVEL 19 — SEARCH SYSTEM

Deeply explain:

- Elasticsearch
- Inverted index
- Tokenization
- Ranking
- Autocomplete
- Fuzzy search
- Filtering
- Faceting
- Sharding
- Replication
- Indexing pipeline.

Design:

- Amazon-like search
- E-commerce search
- Job search.

LEVEL 20 — PERFORMANCE ENGINEERING

Cover:

- Latency
- Throughput
- CPU
- Memory
- Network
- Database optimization
- Query optimization
- Cache optimization
- Connection pooling
- Batching
- Compression
- Async processing
- Load testing.

Teach how to identify bottlenecks during interviews.

LEVEL 21 — INTERVIEW COMMUNICATION

This section is extremely important.

Teach me:

- How to start the interview
- How to clarify requirements
- How to think aloud
- How to communicate trade-offs
- How to handle uncertainty
- What to do when I don't know something
- How to recover if interviewer challenges my design
- How to ask intelligent questions
- How to avoid overengineering
- How to avoid jumping directly into technology choices.

Provide:

Bad answer

Then:

Better answer

Then:

Excellent senior-level answer.

Do this for multiple common interview situations.

LEVEL 22 — SENIOR / STAFF-LEVEL EXPECTATIONS

Because I have 9 years of experience, explain the difference between:

Junior answer

Mid-level answer

Senior answer

Staff-level answer.

Teach me how a senior candidate should demonstrate:

- Trade-offs
- Scalability
- Reliability
- Business understanding
- Cost awareness
- Operational excellence
- Security
- Failure handling
- Evolution of architecture.

LEVEL 23 — COMMON MISTAKES

Create a detailed list of System Design interview mistakes.

Examples:

- Jumping to Kafka immediately
- Using microservices everywhere
- Saying "use Redis" without explaining why
- No capacity estimation
- No requirements clarification
- No failure handling
- Ignoring consistency
- Ignoring security
- Ignoring observability
- Overengineering
- No trade-offs
- Drawing unclear diagrams
- Not explaining bottlenecks.

Explain how to correct each mistake.

LEVEL 24 — RESOURCE LIBRARY

This is extremely important.

Create a categorized resource library containing **direct clickable links**.

A. BEST ENGLISH YOUTUBE CHANNELS

Find the best current System Design channels.

For every resource include:

- Channel
- Playlist/video
- Topic
- Difficulty
- Why it is useful
- Recommended order.

Prioritize high-quality channels such as:

- ByteByteGo
- Gaurav Sen
- Hussein Nasser
- Jordan has no life
- NeetCode where relevant
- Exponent
- freeCodeCamp
- AWS
- Google Cloud
- other genuinely useful channels.

Do NOT blindly include channels just because they are popular.

B. BEST HINDI / HINGLISH YOUTUBE RESOURCES

This is mandatory.

Find the best Hindi/Hinglish System Design lectures.

For each:

- Channel
- Playlist

- Video links
- Topic
- Difficulty
- Recommended sequence.

Prefer instructors who explain actual architecture rather than only superficial interview answers.

C. BEST BLOGS

Include direct links to:

- System Design Primer
- ByteByteGo
- High Scalability
- AWS Architecture Center
- Netflix Tech Blog
- Uber Engineering
- Meta Engineering
- Google Engineering
- Amazon AWS Architecture blogs
- Cloudflare
- LinkedIn Engineering
- DoorDash Engineering
- Airbnb Engineering
- Stripe Engineering
- Martin Kleppmann resources
- other high-quality engineering blogs.

D. BOOKS

Recommend books in order.

Clearly separate:

Must Read

Optional

Advanced

Include:

- Designing Data-Intensive Applications

- System Design Interview
- Database Internals
- Fundamentals of Distributed Systems
- other genuinely valuable books.

Tell me exactly:

> Which chapters should I read and which can I skip?

Do not tell me to read every book completely.

E. COURSES

Compare:

- Free
- Paid
- YouTube
- Udemy
- Educative
- ByteByteGo
- Grokking.

Explain which resource is best for:

- Beginner
- Intermediate
- Interview preparation
- Advanced distributed systems.

LEVEL 25 — 16-WEEK STUDY PLAN

Create a detailed **16-week System Design roadmap**.

Assume approximately:

15 hours/week.

Break down every week into:

- Topics
- Video lectures
- Reading

- Practice
- System design problem
- Revision
- Interview practice.

Example:

Week 1

Networking + HTTP + DNS

Week 2

Scalability + Load Balancer + CDN

...

Continue until Week 16.

LEVEL 26 — DAILY STUDY PLAN

Create:

2-hour/day plan

3-hour/day plan

4-hour/day plan

For each day:

- Learn
- Watch
- Read
- Design
- Revise.

Give me a repeatable daily routine.

LEVEL 27 — SPACED REVISION SYSTEM

Create a revision schedule:

- Same day
- Day 2
- Day 7
- Day 14
- Day 30.

Create a System Design revision checklist.

LEVEL 28 — SYSTEM DESIGN CHEAT SHEET

Create a final compact cheat sheet covering:

- Requirements
- Capacity
- APIs
- DB
- Cache
- Queue
- Scaling
- Consistency
- Reliability
- Security
- Observability
- Trade-offs.

The cheat sheet should be something I can revise before an interview.

LEVEL 29 — INTERVIEW QUESTION BANK

Create at least:

50 Beginner questions

50 Intermediate questions

50 Advanced questions

30 FinTech questions

30 Real-time system questions

30 E-commerce questions

30 Distributed-system questions.

For each question provide:

- Difficulty
- Concepts tested
- Expected interview duration
- What interviewer expects.

LEVEL 30 — MOCK INTERVIEW PROGRAM

Create at least **20 mock System Design interviews**.

Progress:

Mock 1–5

Beginner

Mock 6–10

Intermediate

Mock 11–15

Senior

Mock 16–20

Tier-1 / Amazon / Careem level.

For each mock provide:

- Problem
- Requirements
- Expected solution areas
- Interviewer hints
- Follow-up questions
- Evaluation rubric.

Do NOT reveal the complete solution immediately.

Provide an interviewer version and candidate version.

LEVEL 31 — EVALUATION RUBRIC

Create a scoring system out of 100.

Evaluate:

- Requirement clarification — 10
- Capacity estimation — 10
- Architecture — 15
- Data model — 10
- API design — 5
- Scalability — 15
- Reliability — 10
- Consistency — 5
- Security — 5
- Observability — 5
- Trade-offs — 5
- Communication — 5.

Explain:

0–40 = Not ready

40–60 = Beginner

60–75 = Intermediate

75–85 = Senior interview ready

85–95 = Tier-1 ready

95+ = Excellent.

Adjust these thresholds if research suggests a better model.

LEVEL 32 — FINAL "DESIGN ANY PRODUCT" FRAMEWORK

Create one final framework that I can memorize.

When an interviewer gives me ANY product:

Example:

> "Design Uber."

I should be able to think:

1. Clarify requirements
2. Identify users
3. Identify core workflows
4. Estimate scale
5. Define APIs
6. Define data
7. Draw architecture
8. Identify bottlenecks
9. Add cache
10. Add queue
11. Scale database
12. Handle consistency
13. Handle failures
14. Security
15. Observability
16. Trade-offs
17. Future evolution.

Create a **one-page decision tree** for this.

CRITICAL REQUIREMENT — DO NOT JUST GIVE THEORY

I do NOT want a generic System Design tutorial.

The roadmap must continuously answer:

> "What would I actually say in an interview?"

For every major topic include:

Concept

Simple explanation

Real-world example

Architecture

Trade-off

Interview question

Ideal senior answer

Follow-up question

Common mistake.

RESOURCE QUALITY RULE

Do NOT provide random YouTube videos.

For every video/resource:

- Verify that the link works.
- Prefer authoritative/high-quality resources.
- Prefer complete playlists when available.
- Include Hindi/Hinglish resources separately.
- Include English resources separately.

- Mention if a resource is outdated.
- Avoid duplicate resources teaching the same thing.
- Prioritize resources based on learning value rather than popularity.

Use direct clickable URLs in the final document.

TECHNOLOGY CONTEXT

When examples are needed, prefer technologies relevant to my background:

- PHP/Laravel
- Node.js
- MySQL
- PostgreSQL
- MongoDB
- Redis
- Kafka
- Elasticsearch
- AWS
- Docker
- Kubernetes
- Prometheus
- Grafana
- OpenTelemetry.

However:

Do not force these technologies into every architecture.

Teach me how to choose the correct technology based on requirements.

IMPORTANT ARCHITECTURE DECISION MATRIX

Create tables like:

Requirement	Possible Technology	Why
Low latency cache	Redis	...
Large object storage	S3	...
Async events	Kafka	...
Simple queue	SQS	...

Requirement	Possible Technology	Why
Strong relational transactions	PostgreSQL/MySQL	...
Massive write scale	Cassandra/DynamoDB	...
Full-text search	Elasticsearch	...

Create similar decision matrices for:

- SQL vs NoSQL
- Kafka vs RabbitMQ
- Redis vs Memcached
- REST vs gRPC
- SQL vs MongoDB
- SQS vs Kafka
- ECS vs EKS
- synchronous vs asynchronous communication
- monolith vs microservices
- single-region vs multi-region.

FINAL DELIVERABLES

I want TWO final files:

1. COMPLETE PDF

Create:

System_Design_Master_Roadmap_9YOE.pdf

The PDF must be professional and suitable as my long-term study book.

Include:

- Cover
- Table of contents
- Learning roadmap
- All chapters
- Diagrams
- Tables
- Resource links
- Study schedules
- Interview frameworks
- Cheat sheets

- Practice problems
- Mock interviews
- Company preparation
- Final checklist.

---

2. COMPLETE MARKDOWN FILE

Create:

System_Design_Master_Roadmap_9YOE.md

The Markdown must contain the complete same content as the PDF.

Use:

- Headings
- Tables
- Checklists
- Mermaid diagrams where useful
- Clickable links
- Code blocks for examples
- Interview scripts.

PDF QUALITY REQUIREMENTS

The PDF must be:

- Professionally formatted
- Easy to read
- Proper headings
- Page numbers
- Table of contents
- Tables
- Architecture diagrams
- Callout boxes
- Checklists
- Interview tips
- Good spacing
- No broken links
- No overlapping text
- No cut-off content.

Do not create a superficial 20–30 page PDF.

This should be a **comprehensive long-term System Design study guide**.

If the content becomes very large, prioritize completeness and structure over artificially keeping the PDF short.

LEARNING PHILOSOPHY

Follow this progression:

Understand → Visualize → Implement mentally → Design → Explain → Defend → Optimize

Do not make me memorize architecture diagrams.

Teach me the reasoning behind them.

The ultimate skill I need is:

> Given an unfamiliar product and a set of requirements, I should be able to derive the architecture logically rather than memorize a predefined solution.

FINAL OUTPUT STRUCTURE

At the very beginning provide:

My System Design Journey

```
Fundamentals
      ↓
Networking
      ↓
Scalability
      ↓
Databases
      ↓
Caching
      ↓
Messaging
      ↓
Distributed Systems
      ↓
Microservices
```

```
      ↓
Cloud
      ↓
Reliability
      ↓
Security
      ↓
Observability
      ↓
Architecture Patterns
      ↓
System Design Problems
      ↓
Mock Interviews
      ↓
Company-Specific Preparation
      ↓
Amazon / Careem / Tier-1 Ready
```

Then provide:

1. Current skill assessment

Based on my background, explain what I probably already know and what I need to learn.

2. Complete roadmap

3. Topic-by-topic curriculum

4. Resource library

5. 16-week plan

6. Daily plan

7. System Design interview framework

8. 30+ complete design problems

9. Company-specific preparation

10. Mock interviews

11. Cheat sheets

12. Final readiness checklist

FINAL SUCCESS CRITERIA

At the end of the roadmap, I should be able to confidently answer:

- Design URL Shortener
- Design WhatsApp
- Design Instagram
- Design YouTube
- Design Netflix
- Design Uber
- Design Careem
- Design Amazon
- Design Food Delivery
- Design Payment Gateway
- Design Wallet
- Design Notification System
- Design Search
- Design Rate Limiter
- Design Distributed Cache
- Design Ride Matching
- Design Ticket Booking
- Design E-commerce
- Design Real-time Location Tracking.

More importantly, I should be able to handle an **unfamiliar system design question** using first principles.

IMPORTANT FINAL INSTRUCTION

Do not stop at explaining the roadmap.

Actually create the files:

1. **System_Design_Master_Roadmap_9YOE.pdf**
2. **System_Design_Master_Roadmap_9YOE.md**

Before finalizing, verify that:

- All major HLD topics are covered.
- Beginner → advanced progression is logical.
- Resources are current and clickable.
- Hindi and English resources are separated.
- Interview strategy is included.
- Amazon/Careem-level preparation is included.
- Practical system-design problems are included.
- The PDF and Markdown contain the complete roadmap.

- There are no major gaps in distributed systems, databases, scalability, reliability, security, observability, or cloud architecture.

Do not give me only an outline.

Generate the complete study material and both final files.